

Manifeste Ibude - Architecture & Apprentissage



Frontend & Client

web/

Notion apprise : Développement frontend moderne (React/Vue/Next.js), State management, API consumption

Partie du projet : Interface utilisateur principale

Tâche : Afficher les annonces, gérer les réservations, profils utilisateurs, paiements

Quand : Dès le début - c'est la vitrine de ton projet

mobile/

Notion apprise : Développement mobile cross-platform (React Native/Flutter)

Partie du projet : Application mobile iOS/Android

Tâche : Version mobile de la plateforme, notifications push, géolocalisation

Quand : Phase 2 - une fois le web stabilisé



Backend & Services

backend/go/

Notion apprise : Go, microservices, API REST/GraphQL, concurrence

Partie du projet : Service principal (API gateway, business logic)

Tâche : Gestion des utilisateurs, annonces, réservations, authentification

Quand : Dès le début - le cœur de l'application

backend/rust/

Notion apprise : Rust, performance, safety, WebAssembly

Partie du projet : Microservices critiques (paiements, recherche)

Tâche : Traitement haute performance, système de recherche, moteur de recommandations

Quand : Phase 2 - pour optimiser les parties critiques

backend/shared/

Notion apprise : Code réutilisable, protocoles de communication inter-services

Partie du projet : Bibliothèques communes entre Go et Rust

Tâche : Types de données partagés, utilities, contrats d'API (protobuf)

Quand : Quand tu as plusieurs services qui communiquent



Intelligence Artificielle

ml/

Notion apprise : Machine Learning, Python, scikit-learn, TensorFlow/PyTorch

Partie du projet : Intelligence de la plateforme

Tâche :

- Pricing dynamique (ajuster les prix selon la demande)
- Système de recommandations (suggérer des logements)
- Détection de fraude
- Analyse de sentiments sur les avis

Quand : Phase 2 - une fois que tu as des données



Données & Persistance

database/migrations/

Notion apprise : SQL, gestion de schéma de base de données, versioning

Partie du projet : Structure de la base de données

Tâche : Créer et modifier le schéma (tables users, listings, bookings, payments)

Quand : Dès le début - avant même de coder le backend

database/seeds/

Notion apprise : Génération de données de test, fixtures

Partie du projet : Données de développement

Tâche : Peupler la DB avec des utilisateurs, annonces et réservations fictives

Quand : Dès que les migrations sont prêtes



Conteneurisation & Orchestration

docker/

Notion apprise : Docker, containerisation, multi-stage builds

Partie du projet : Environnement de développement et production

Tâche : Dockerfiles pour chaque service, docker-compose pour orchestrer localement

Quand : Dès le début - pour un environnement reproductible

infrastructure/kubernetes/

Notion apprise : Kubernetes, orchestration de containers, scaling

Partie du projet : Déploiement en production

Tâche : Deployments, Services, Ingress, ConfigMaps, Secrets pour chaque microservice

Quand : Phase 3 - pour déployer en production de manière scalable

infrastructure/terraform/

Notion apprise : Infrastructure as Code, cloud providers (AWS/GCP/Azure)

Partie du projet : Provisionnement de l'infrastructure cloud

Tâche : Créer clusters Kubernetes, bases de données managées, load balancers, CDN

Quand : Phase 3 - avant de déployer sur le cloud

infrastructure/ansible/

Notion apprise : Configuration management, automatisation

Partie du projet : Configuration des serveurs

Tâche : Installer et configurer les services, déployer les applications

Quand : Optionnel - si tu gères tes propres serveurs



Monitoring & Observabilité

monitoring/prometheus/

Notion apprise : Métriques, monitoring temps réel

Partie du projet : Surveillance de la santé des services

Tâche : Collecter métriques (CPU, RAM, requêtes/sec, latence) de tous les services

Quand : Phase 2 - dès que tu déploies

monitoring/grafana/

Notion apprise : Visualisation de données, dashboards

Partie du projet : Tableaux de bord de monitoring

Tâche : Créer des dashboards pour visualiser les métriques Prometheus

Quand : Juste après Prometheus

monitoring/alerts/

Notion apprise : Alerting, SRE practices

Partie du projet : Système d'alertes

Tâche : Définir des règles d'alerte (service down, CPU élevé, erreurs 500)

Quand : Avec Prometheus/Grafana



Configuration

configs/nginx/

Notion apprise : Reverse proxy, load balancing, SSL/TLS

Partie du projet : Gateway HTTP

Tâche : Router les requêtes vers les bons services, gérer HTTPS, cache statique

Quand : Dès que tu as plusieurs services

configs/redis/

Notion apprise : Cache in-memory, sessions, pub/sub

Partie du projet : Cache et sessions utilisateurs

Tâche : Cacher les résultats de recherche, stocker les sessions, file d'attente

Quand : Phase 2 - pour améliorer les performances

`configs/postgres/`

Notion apprise : Base de données relationnelle, optimisation SQL

Partie du projet : Base de données principale

Tâche : Configuration PostgreSQL (pools de connexion, réplication, backup)

Quand : Dès le début

Tests

`tests/e2e/`

Notion apprise : Tests end-to-end (Cypress, Playwright)

Partie du projet : Tests du parcours utilisateur complet

Tâche : Tester les scénarios : inscription → recherche → réservation → paiement

Quand : Phase 2 - une fois les fonctionnalités principales implémentées

`tests/integration/`

Notion apprise : Tests d'intégration entre services

Partie du projet : Tests inter-services

Tâche : Vérifier que backend Go + Rust + DB + Redis fonctionnent ensemble

Quand : Dès que tu as plusieurs services

`tests/load/`

Notion apprise : Tests de performance (k6, Locust)

Partie du projet : Tests de charge

Tâche : Simuler 1000 utilisateurs simultanés, identifier les bottlenecks

Quand : Phase 3 - avant la mise en production

CI/CD

ci-cd/github/

Notion apprise : GitHub Actions, pipelines CI/CD

Partie du projet : Automatisation du build et déploiement

Tâche : À chaque push : tests → build → deploy automatique

Quand : Dès le début - pour automatiser ton workflow

ci-cd/jenkins/

Notion apprise : Jenkins, pipelines complexes

Partie du projet : Alternative à GitHub Actions

Tâche : Pipelines plus complexes avec approval manuelle

Quand : Optionnel - si GitHub Actions ne suffit pas

Utilitaires

scripts/dev/

Notion apprise : Bash/Shell scripting, automatisation

Partie du projet : Faciliter le développement local

Tâche : Scripts pour setup initial, reset DB, seed data, générer données de test

Quand : Dès le début - ton premier script = setup.sh

scripts/deploy/

Notion apprise : Déploiement automatisé

Partie du projet : Déploiement en prod

Tâche : Scripts pour déployer sur différents environnements (staging, prod)

Quand : Phase 3 - avant le premier déploiement

scripts/backup/

Notion apprise : Stratégies de backup, disaster recovery

Partie du projet : Sauvegardes

Tâche : Backup automatique de la DB, restauration, rotation des backups

Quand : Dès la mise en production

[tools/cli/](#)

Notion apprise : Développement d'outils CLI (Cobra en Go, Clap en Rust)

Partie du projet : Outil de gestion du projet

Tâche : CLI custom pour gérer Ibude ([ibude start](#), [ibude migrate](#), [ibude seed](#))

Quand : Phase 2 - pour faciliter les opérations



Documentation

[docs/api/](#)

Notion apprise : Documentation d'API (OpenAPI/Swagger)

Partie du projet : Documentation technique

Tâche : Documenter tous les endpoints REST/GraphQL, schémas de données

Quand : Au fur et à mesure du développement

[docs/architecture/](#)

Notion apprise : Architecture diagrams (C4, UML)

Partie du projet : Documentation système

Tâche : Schémas d'architecture, diagrammes de séquence, choix techniques

Quand : Dès le début - pour clarifier ta vision

[docs/guides/](#)

Notion apprise : Rédaction technique

Partie du projet : Guides pour développeurs

Tâche : Comment setup le projet, contribuer, déployer, debugging

Quand : Au fur et à mesure

Roadmap d'apprentissage recommandée

Phase 1 - Foundation (Mois 1-2)

1. `database/` - Modéliser ton schéma
2. `backend/go/` - API REST basique
3. `web/` - Interface simple
4. `docker/` - Containeriser le tout
5. `scripts/dev/` - Automatiser le setup
6. `ci-cd/github/` - Pipeline basique

Phase 2 - Features (Mois 3-4)

1. `backend/rust/` - Microservice de recherche
2. `configs/redis/` - Ajouter du cache
3. `tests/integration/` - Tester les services
4. `ml/` - Premiers modèles
5. `monitoring/` - Observer ce qui se passe

Phase 3 - Production (Mois 5-6)

1. `infrastructure/terraform/` - Provisionner le cloud
 2. `infrastructure/kubernetes/` - Déployer
 3. `tests/load/` - Tester la charge
 4. `scripts/backup/` - Sécuriser les données
 5. `mobile/` - Étendre aux mobiles
-



Conseil

Ne cherche pas à tout faire d'un coup ! Commence par un MVP (Minimum Viable Product) avec juste `backend/go/`, `web/`, `database/` et `docker/`. Le reste viendra naturellement quand tu en auras besoin.